Affordable Mistakes: Severity-Aware Multiparty Session Types for Participants that Choose Wrongly

Anonymous author
Anonymous affiliation

Anonymous author
Anonymous affiliation

Abstract

Multiparty session types guarantee that well-typed participants interact correctly, on the hypothesis that each participant *is* its type. A participant that was instructed rather than compiled — a language-model agent — breaks that hypothesis in one specific way: at a branch where it should select A it may select B. Forbidding this is impossible, and would be useless if it were possible. We change what the discipline is for: it does not prevent wrong choices, it prevents catastrophic ones and reports the rest as risk.

The fault model is the participant's own selection at a branch, defined through the guards that asserted global types already carry. Cascading is a budget k of wrong selections, with no probabilities anywhere. Budgeted deviation is itself old — reactive synthesis and robustification have owned it for a decade. What is new is carrying a quantity a participant *spends* across a reduction relation and into typed programs: a session typed against a k-tolerant protocol by the base discipline's conformance judgment is hazard-free on every run of cost at most k, budgets distribute over participants, and the guarantee survives recursion and every permutation of independent nodes. One syntax-directed rule characterizes the condition exactly; each wrong selection is classified Benign, Futile or Catastrophic by outcome against a STRIPS-style world, and these are three consecutive intervals of budgets, not an arbitrary carve-up. The condition composes against an interface rather than a protocol's internals, and is decidable for regular protocols. Everything is mechanized in Coq, axiom-free; the tool's own search is not extracted, but on the boolean fragment its verdict is cross-checked against a kernel that is.

The evaluation asks whether any of this matters to a real agent. On a benchmark of seventeen protocols the analysis names the point-of-no-return action of every 0-tolerant one; 340 runs of two production models find every observed catastrophe consistent with the theorem, and six of the nineteen Catastrophic verdicts taken by a live agent; a token-free census of 162 skills from thirteen public repositories with its false-refutation rate audited by hand; and 134 agent runs in two runtimes over 16 documents — eight of those skills and eight specification cases of our own — every artifact verified by recomputation, in which the refusals coincide with the runs that ended in wasted tokens or a fabricated result: at realistic input sizes no refuted run produced a correct artifact.

2012 ACM Subject Classification Theory of computation → Type structures; Theory of computation → Process calculi; Software and its engineering → Software verification

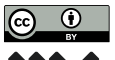
Keywords and phrases Session types, Multiparty, Behavioural types, Safety, Irreversibility, Fault tolerance, Mechanized metatheory

Digital Object Identifier 10.4230/LIPIcs.ECOOP..0

Supplementary Material Anonymous supplementary material

1 Introduction

Multiparty session types (MPST) guarantee communication safety, session fidelity and deadlock-freedom for systems whose participants *are* their local types [1]: the code was



© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0



checked, so every run is a compliant run. A participant that was instructed rather than compiled breaks that hypothesis in a specific way. At a choice point it may take branch B where the situation demanded branch A — not by violating the protocol (both branches are in the type) but by judging the situation wrongly. This is the ordinary condition of a language-model agent occupying a role, and it is why such agents are employed: their latitude.

Nothing here is specific to language models. The hypothesis MPST rests on — the participant *is* its type — fails for every participant that was instructed rather than compiled: an operator following a runbook at a migration console, a third-party service behind an adapter whose behaviour is a contract rather than a checked artifact, a learned controller. Each occupies a role and may, at a branch the protocol offers, take the one the situation forbids. Language-model agents are the case that makes the question urgent and the case our evaluation can drive at scale, but the discipline asks only that the participant choose, not how.

The tempting response is to make the type system prevent it. That is doubly wrong: impossible, because the judgement is the participant's, not the protocol's; and undesirable, because a participant that may never choose wrongly is one that may never choose. What is needed is *triage*.

The reframing. Do not ask whether the participant complied. Ask *what a wrong choice costs*. Three answers matter, and the middle one is what existing disciplines cannot express (Figure 1):

- Benign — a detour; the goal is still reachable.
- Futile — the goal is lost, but nothing is harmed. *Failure is not disaster.*
- Catastrophic — a hazard becomes reachable.

A system that blocks Futile is useless; one that permits Catastrophic is dangerous. Separating them is the whole job. Thesis: *session types say what may happen; this says what you can survive.*

What is and is not new. The ingredients are old; the discipline is not. Branch guards on global types are design-by-contract MPST [2, 4, 5], and guard violation on selection is the alarm condition of the monitoring line [3]. Whether a goal remains reachable is dead-end detection in planning [28, 29]; whether an effect can be undone is undoability checking [30, 31]. Bounded fault tolerance is classical in planning [26, 27] and in MPST with crash-stop failures [8, 9]. A syntax-directed system equivalent to a model checker is a known template [20, 21]. We claim none of these. What we claim is C1–C5 below, and nothing wider.

We are careful with the word *type system*. The condition is on global types, as well-assertedness [2] and deadlock-freedom conditions are; the participants are typed by the base discipline's direct-conformance judgment, unchanged. The type-theoretic content is the theorem that couples the two (Section 7), not the rule that decides the condition.

Contributions. Throughout, ✓`name` marks a result proved in the accompanying Coq development and names its identifier there.

C1 *A budget a typed participant spends, carried to programs* (§4, §7). A session typed against a k-tolerant protocol is hazard-free on every run whose misselection cost is at most k (✓`bridge_run`); the budget distributes over participants, so a global allowance is a contract that can be split and checked per role (✓`budget_distributes`); the guarantee

survives recursion (✓ `bridge_mu`) and every permutation of independent nodes, with the side conditions discharged syntactically (✓ `bridge_interleaved`, ✓ `strips_swap_act`). Budgeted deviation is not ours (§12); *carrying a quantity a participant spends across a reduction relation, into typed programs*, is what no prior discipline does.

C2 *The condition that makes C1 checkable* (§6): one syntax-directed rule, sound and complete for k-tolerance (✓ `TC_exact`), which is exactly non-reachability in a product graph (✓ `TC_regular`). Read coinductively it is the same condition on recursive protocols (✓ `TR_exact`, ✓ `TC_is_TR`), so the rule, the decision procedure and the bridge are about one object at both layers; and the tolerance degree k^*, the largest budget a protocol survives, is principal: the budgets it tolerates are exactly those $\leq k^*$ (✓ `principal_characterises`).

C3 *An ordered diagnosis* (§5): every wrong choice is Benign, Futile or Catastrophic, and these are not an arbitrary carve-up — the class of a residual is monotone in the budget along the chain Futile $<$ Benign $<$ Catastrophic (✓ `severity_monotone_in_budget`), so the three classes are consecutive intervals of budgets and k^* is a genuine threshold (✓ `tolerance_degree_is_a_threshold`). Budgeted reachability is exactly "some run with at most k wrong choices reaches a hazard" (✓ `reach_mu_iff_run`), so the classes are statements about executions. Benign is a possibility, not a guarantee, and the guarantee is separated from it and shown strictly stronger (✓ `robust_benign`, ✓ `benign_is_not_robust`).

C4 *Decidability and modularity* (§8, §9): the μ-unfolding closure is finite (✓ `cands_closed`) and the decision procedure is proved sound and complete (✓ `decide_mu_correct`); sequential composition is typed against an interface that carries the remaining budget, not the internals (✓ `TC_seq_interface`), and that interface may be projected onto the cone of influence of what remains (✓ `interface_projection`), which is what makes modular checking cheap rather than merely decomposable. Four repairs are proved sound, two exactly, and the condition is a congruence, so they apply where the tool applies them (✓ `safeT_congruence`).

C5 *Evidence* (§10): an implementation whose verdict agrees with a kernel extracted from the proof on 500 of 500 random protocols; a severity benchmark; 340 live-agent runs showing the Catastrophic verdicts name branches real agents take; a token-free census over 162 real skills with its false-refutation rate audited by hand; and 134 agent runs in two runtimes over 16 documents — eight of those skills and eight specification cases of our own — showing the achievability verdicts predict which runs are wasted or fabricated.

Every theorem above is checked for vacuity, because the predecessor of this discipline was withdrawn when its own audit found it had none (§13): conforming sessions are exhibited for the protocols the paper reasons about and for their repairs, the guard repair's runtime is built and shown to have the worlds its theorem needs, and the cone-of-influence and congruence theorems are instantiated where their hypotheses could have been satisfiable nowhere (✓ `cone_is_not_degenerate`, ✓ `congruence_is_not_degenerate`), and Robust, the class the paper says is strictly stronger than Benign, is shown non-empty (✓ `Ggood_is_robust`).

A design expectation was refuted by the mechanization and a benchmark finding refines it: tolerance is anti-monotone in the capability context for a fixed hazard (✓ `tolerance_antitone_in_ctx`), yet a new tool that re-establishes a lost atom can *remove* a point of no return (§10).

2 Overview

Running example. A planner `p` and a worker `q` holding a purchase tool. Atoms `verified`, `booked`, `funds`; `verify` adds the first, `purchase` adds the second and deletes the third, and

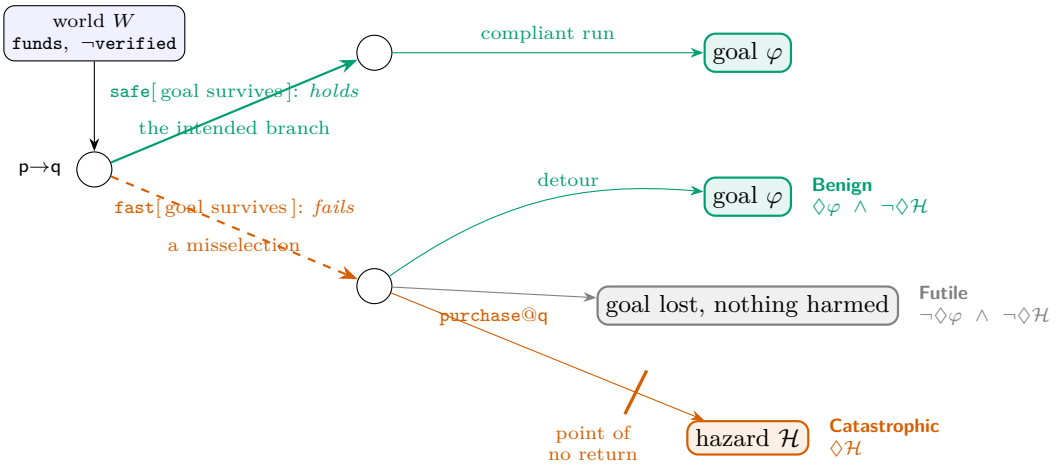


Figure 1 The idea, on the running example of §2. At a guarded choice in world W the branch whose guard holds is intended; taking another is a misselection. Its severity is a property of the residual, and the three fates drawn are the only three: the goal still reachable (Benign), the goal lost with no hazard reachable (Futile), or a hazard reachable (Catastrophic) — here by crossing a point of no return, an irreversible capability after which the goal cannot be recovered. In G_{bad} the misselected branch takes the third.

no tool restores `funds`. The protocol offers two branches:

$$G_{\mathsf{bad}} \;=\; \mathsf{p}{\to}\mathsf{q} : \{\; \texttt{safe}.\ \texttt{verify@q}.\ \texttt{purchase@q}.\ \mathsf{End}\ ,\ \ \texttt{fast}.\ \texttt{purchase@q}.\ \mathsf{End}\ \}$$

with goal $\varphi = \texttt{booked} \wedge \texttt{verified}$. Under the default instantiation (§4) `safe` is intended because the goal survives it and `fast` is a misselection because it does not. The tool reports, and the Coq development proves for the same instance: G_{bad} is 0-tolerant (✓ `Gbad_is_0_tolerant`) — if the participant never misselects, nothing bad happens, which is where classical MPST stops; it is *not* 1-tolerant (✓ `Gbad_not_1_tolerant`) — one wrong choice purchases unverified, and `purchase` is the point of no return; and narrowing away `fast` is tolerant at every k (✓ `Ggood_is_k_tolerant`). The headline number is the *tolerance degree* k^*, the largest k the protocol survives.

For a protocol with two decision points the tool's output is what an engineer needs (Section 10, `migration_backup`):

```
migration_backup: tolerance degree k* = 1
  choice nodes 4, branches 6, irreversible ['drop_old']
  /choice@ops#0
    backup              Benign        intended
    skip_backup         Catastrophic  misselection  PNR=drop_old
  /choice@ops#0/backup/choice@ops#2
    drop_old            Benign        intended
    keep_old            Benign        intended
  /choice@ops#0/skip_backup/choice@ops#1
    drop_old            Catastrophic  misselection  PNR=drop_old
    keep_old            Futile        misselection
  first catastrophe within budget 2:
    choose/skip_backup > act/migrate > choose/drop_old > act/drop_old
  repair (narrow):
    remove skip_backup at /choice@ops#0
    remove drop_old at /choice@ops#0/skip_backup/choice@ops#1
  bystander interleavings: exact (0 independent pairs)
```

"Tolerant of one wrong choice; two wrong choices reach data loss, here is the path and the action that burns the bridge" is actionable. "Impossible" is not.

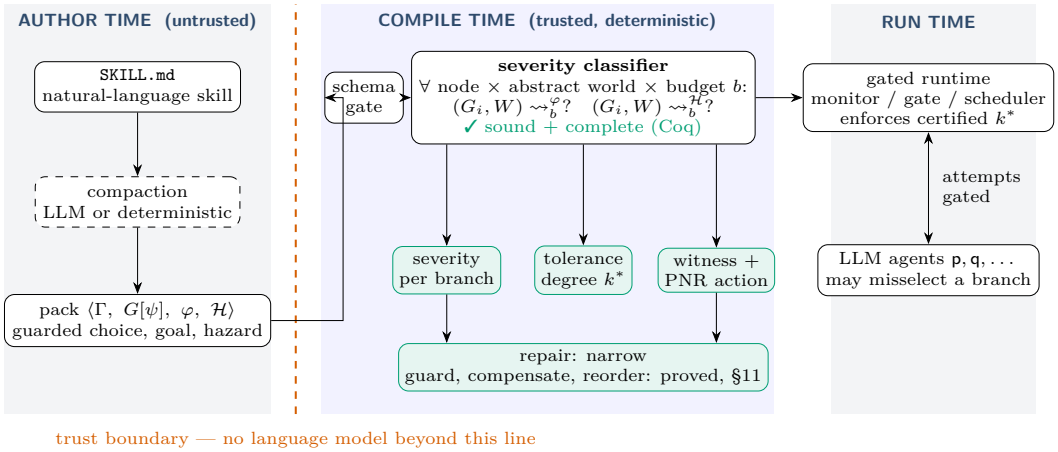


Figure 2 Architecture. Compaction from prose to a pack is untrusted and may use a language model; nothing beyond the trust boundary does. The severity classifier runs the discipline's existing reachability search pointwise at every branch of every choice, at every abstract world and budget, and emits a per-branch severity, the tolerance degree k^*, a witness naming the point-of-no-return action, and the narrowing repair. The certified pack drives the gated runtime that refuses out-of-protocol attempts.

Architecture. Figure 2: a compile-time check on the pack, before any participant runs, with no language model in the trusted core — the base discipline's trust boundary. The runtime gate is inherited unchanged: it refuses *out-of-set* attempts (labels the protocol does not offer) before delivery, so those change no state. The deviation this paper analyses is *in-set*: a branch the type permits and the situation forbids.

3 The Base Discipline

We build on a goal-marked, capability-guarded synchronous multiparty discipline, stated here in the detail this paper uses. Predicates $p \in \mathsf{Pred}$, numeric variables $x \in \mathsf{Var}$, roles p, q, capabilities $a \in \mathsf{Cap}$, labels $\ell \in \mathsf{Lab}$. The state logic is quantifier-free linear integer arithmetic with uninterpreted booleans; a world $W = \langle B, N \rangle$ valuates it, and the checker's decidable fragment abstracts predicates concretely and numerics symbolically, widening at loop heads so the reachable world set is finite. A capability context maps each possessed tool to a guarded STRIPS effect $\Gamma(a) = \langle \mathit{pre}_a, \mathit{eff}_a \rangle$, where eff_a adds and deletes predicates and assigns numerics. Firing is World-Act: $\langle W \rangle\, a\, \langle W' \rangle$ iff $W \vDash \mathit{pre}_a$ and $W' \in \mathrm{apply}(\mathit{eff}_a, W)$. We write φ for the goal and $\mathcal{H}$ for the hazard, both predicates over worlds. Global types are coinductive regular trees

$$G \;::=^{coind}\; \mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i \in I} \;\mid\; a@\mathsf{p}.\, G \;\mid\; \checkmark\varphi.\, G \;\mid\; \mathsf{End}$$

and sessions $\mathbb{M} = \mathsf{p}_1[P_1] \parallel \cdots$ of processes $P ::=^{coind} \mathbf{0} \mid \mathsf{p}!\{\ell_i.P_i\} \mid \mathsf{p}?\{\ell_i.P_i\} \mid a.P$ are typed *directly* against G by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — no local types, projection, or separate subtyping [16, 17] — whose T-Comm requires the sender to offer exactly the protocol's labels, the receiver at least them, and every branch to check against the same residual. A *pack* $\langle \Gamma, G, \varphi, \mathcal{H} \rangle$ is the checked object: a capability context, a global type, a goal and a hazard. Achievability is existential may-reachability of a goal-satisfying configuration; the base metatheory (subject reduction, session fidelity, refutation soundness) is mechanized

for the finite fragment. This paper adds one annotation to choice, one counter to the semantics, and one well-formedness condition.

4 Misselection and the Budget

▶ **Definition 1** (Guarded choice). *A choice node carries, per branch, the predicate that makes that branch the intended one:* $\mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].\, G_i\}_{i\in I}$. *In world* W, *branch* i *is* intended *if* $W \vDash \psi_i$; *taking a branch with* $W \nvDash \psi_i$ *is a* misselection.

The syntax is that of asserted global types [2], whose well-formedness conditions guarantee that a compliant participant can always pick a satisfied branch. We keep the syntax and drop the guarantee: the guards are not a constraint on an implementation but the *definition of wrong*.

Default instantiation. Guards need not be written. When a branch carries none, the implementation uses the *rational-choice* default: a branch is intended in W iff the goal is still reachable from its residual. Under this default a misselection is a choice that forfeits the goal, and once the goal is forfeit every subsequent choice is a misselection — there is no longer a right one — so the budget counts decisions taken after the point at which the task was lost. Explicit guards override the default where the intended branch is a matter of policy rather than reachability (Section 10 uses both).

▶ **Definition 2** (Budgeted reachability). $(G, W) \rightsquigarrow_b^{\mathcal{H}}$ *holds when a hazard state is reachable from* (G, W) *using at most* b *misselections (Coq:* `reach_haz`*): the head rules of the base semantics, plus*

$$\textit{RH-Comm-Ok}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \vDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_b^{\mathcal{H}}}$$

$$\textit{RH-Comm-Dev}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \nvDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_{b+1}^{\mathcal{H}}}$$

Compliant progress is free; a misselection costs one unit. A choice may instead be marked as controlled by the environment, in which case every branch is taken demonically at no cost; the implementation supports this and one benchmark protocol uses it, but the mechanization covers only participant-controlled choice, so no theorem in this paper is stated for it.

▶ **Definition 3** (k-misselection tolerance). G *is* k-misselection-tolerant *for hazard* $\mathcal{H}$ *under* Γ *from* W_0 *if* $\neg\,(G, W_0) \rightsquigarrow_k^{\mathcal{H}}$.[1] 0*-tolerance is classical MPST safety.*

The budget is possibilistic: a design parameter of the deployment, not a measurement of the participant. Every theorem below holds for every participant that misselects at most k times, however it does so.

[1] We avoid two established names. In MPST, *k-safety* is a buffer bound [14]; in hyperproperties it is a property of k traces [15]. *k-resilient* plans survive k action failures [26]; our fault is the participant's own selection and our target is a hazard, not the goal.

5 Severity and the Point of No Return

Let φ be the goal and $\mathcal{H}$ the hazard; $\rightsquigarrow_b^{\varphi}$ is budgeted goal reachability.

▶ **Definition 4** (Severity). *For a residual (G, W) at budget b: Catastrophic if $(G, W) \rightsquigarrow_b^{\mathcal{H}}$; otherwise Futile if $\neg(G, W) \rightsquigarrow_b^{\varphi}$; otherwise Benign.*

The trichotomy is not an arbitrary carve-up of two predicates. Both reachability questions are monotone in the budget (✓ `reach_mono_budget`), and that makes the classes *ordered*.

▶ **Assumption 5** (Decidable questions). *Where a result needs to know whether a guard holds, or whether a hazard or the goal is reachable at a given budget, that question is assumed decidable. The widened finite-range fragment of §3 supplies it. In Coq the disjunction is a hypothesis of the affected lemmas, never an axiom, so the development is axiom-free; but we do not oversell that. Where the hypothesis is stated over an arbitrary world predicate it is excluded middle for that class, discharged by the caller rather than assumed globally, and a reader should judge those lemmas as classical in content and constructive only in bookkeeping.*

▶ **Theorem 6** (The trichotomy is ordered). *Rank Futile < Benign < Catastrophic. The rank of a residual is monotone non-decreasing in its budget (✓ `severity_monotone_in_budget`): raising the budget moves a Benign residual to Benign or Catastrophic and never to Futile (✓ `benign_step_up`, ✓ `benign_no_regress`), lowering it moves a Benign residual to Benign or Futile (✓ `benign_step_down`); Catastrophic is upward closed (✓ `catastrophic_upward`) and Futile downward closed (✓ `futile_downward`). So the budgets at which a residual is Futile, then Benign, then Catastrophic are three consecutive intervals of $\mathbb{N}$, in that order, any of them possibly empty.*

Two consequences. First, k^* is a genuine *threshold* and not merely the budget at which a search stopped: if no hazard is affordable at k and one is at $k+1$, then none is affordable at any $b \leq k$ and one is at every $b > k$ (✓ `tolerance_degree_is_a_threshold`), which is what licenses computing it by a scan, and below the threshold every residual is Benign or Futile (✓ `severity_classes_are_separated`). Second, the classes are about *executions*: budgeted reachability holds exactly when some run of the protocol with at most k misselections ends in a hazard state (✓ `reach_mu_iff_run`), so Catastrophic means "an affordable run reaches harm", not merely "a predicate holds".

▶ **Theorem 7** (Partition). *The classes are pairwise disjoint, and exhaustive whenever the two reachability questions are decidable.* ✓ `severity_disjoint`, ✓ `severity_exhaustive`

The content is the separation of Futile from Catastrophic. In model-based safety assessment [34] severity is supplied by the analyst; here it is computed from goal reachability.

The two quantifiers. Catastrophic and Benign are both existential over affordable runs, and that means different things on the two sides. For the hazard it is the reading one wants: a possible catastrophe is a catastrophe. For the goal it says only that the goal is still *possible*, and possible over the future choices of exactly the participant the discipline says cannot be relied on. So Benign must not be read as a guarantee, and we give the guarantee its own name. Say (G, W) *assures* φ at budget b when every affordable run arrives — every intended branch delivers, so does every misselection the budget can pay for, and so does every answer the environment may give — and call (G, W) Robust when it is hazard-free and assures the goal.

▶ **Theorem 8** (Assurance is strictly stronger). *What is assured is reachable (✓ `assured_reach`), so Robust implies Benign (✓ `robust_benign`); and assurance is downward closed in the budget (✓ `assured_downward`), since more affordable mistakes is a stronger obligation — the opposite direction from Catastrophic. The implication is strict: a protocol whose safe branch reaches the goal and whose misselectable branch harmlessly does nothing is Benign at budget* 1 *and not Robust (✓ `benign_is_not_robust`). One affordable mistake costs the goal and nothing else, which is exactly what the existential reading hides.*

The evaluation reports Benign, because that is what the tool computes; Robust is the stronger verdict a tool could compute on the same graph and we have not built it.

▶ **Definition 9** (Point of no return). (G, W) *is a point of no return for* φ *if* $\neg(G, W) \rightsquigarrow_b^{\varphi}$. *An action is* goal-destroying *at* (G, W) *if the goal is reachable before it fires and at no successor.*

Default hazard. Because effects are STRIPS, an atom in Del_a is restored only by some b with the atom in Add_b; if Γ has no such b, a is *irreversible*. The implementation's default hazard, needing no annotation, is *burning a bridge while lost*: an irreversible action fires from a configuration after which the goal is unreachable. Tools whose real-world effect cannot be undone but whose pack models no delete-list (an email cannot be unsent) are declared irreversible explicitly. This is dead-end detection [28] and undoability checking [30] lifted to a protocol; unrestricted reversibility checking is harder than planning [31], and our decidability rests on the widened finite-range fragment the checker already commits to.

6 The Well-Formedness Condition, Exactly

$\vdash_b G, W$ reads: from (G, W), no hazard arises under any run with at most b misselections.

$$\text{T-End}\ \frac{W \not\models \mathcal{H}}{\vdash_b \mathsf{End}, W} \qquad \text{T-Goal}\ \frac{W \not\models \mathcal{H} \qquad \vdash_b G, W}{\vdash_b \checkmark\varphi.G, W}$$

$$\text{T-Act}\ \frac{W \not\models \mathcal{H} \qquad \forall W'.\ \langle W\rangle\, a\, \langle W'\rangle \implies \vdash_b G, W'}{\vdash_b a@\mathsf{p}.G,\ W}$$

$$\text{T-Choice-Safe}\ \frac{\begin{array}{c} W \not\models \mathcal{H} \\ \forall i \in I.\ \ W \models \psi_i \implies \vdash_b G_i, W \\ \forall i \in I.\ \ W \not\models \psi_i \implies \forall c.\ b = c{+}1 \implies \vdash_c G_i, W \end{array}}{\vdash_b\ \mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].G_i\}_{i\in I},\ W}$$

An intended branch is checked at the same budget; a misselectable branch at one less, and not at all when the budget is exhausted. This is "affordable mistake" as a rule.

▶ **Theorem 10** (Exact characterization). $\vdash_k G, W \iff \neg(G, W) \rightsquigarrow_k^{\mathcal{H}}$, *hence a branch is Catastrophic iff its residual fails the condition.* ✓ `TC_sound`, ✓ `TC_complete`, ✓ `TC_exact`

The rule on the running example. Take G_{bad} at $b = 1$ in $W_0 = \{\texttt{funds}\}$. T-Choice-Safe checks the intended branch `safe` at budget 1 and the misselectable branch `fast` at budget 0. The safe branch verifies then purchases and no intermediate world is a hazard, so it holds. The fast branch must satisfy $\vdash_0 \texttt{purchase}@\mathsf{q}.\mathsf{End}$, W_0: by T-Act that needs $\vdash_0 \mathsf{End}, W'$ for every W' with $\langle W_0\rangle\, \texttt{purchase}\, \langle W'\rangle$, and every such W' has `booked` without `verified` — the hazard. The premise fails, so $\not\vdash_1 G_{\mathsf{bad}}, W_0$, and by Theorem 10 a hazard is reachable

within one misselection. At $b = 0$ the misselectable branch carries no obligation at all and the derivation goes through: G_{bad} is 0-tolerant and not 1-tolerant, which is what the tool reports and what ✓ `Gbad_is_0_tolerant` and ✓ `Gbad_not_1_tolerant` prove. Two protocols carry the paper: G_{bad} is the smallest that exhibits the question, and `migration_backup`, whose output §2 shows, is the smallest that exhibits *cascading* — one wrong choice is affordable, two are not.

The rule is not deep and we do not claim it is: it is the reachability relation with the negation pushed through, and the proof is short. What matters is what exactness buys, and it is three things used later. A refutation is never an artifact of conservative rules, so the refutation asymmetry the discipline is built on survives. The condition is syntax-directed, which is what makes composition against an interface possible (§9) instead of whole-system analysis. And because it is a judgment on (G, W) rather than a decision procedure over a graph, it is the interface through which Theorem 13 couples the budget to typed programs: a decision procedure alone does not compose with a typing derivation.

Two admissible rules. The condition has the structural properties a type system is expected to have, and they are already proved. Budgets *subsume*: a derivation at k is a derivation at every smaller budget, so

$$\frac{\vdash_k G, W \qquad k' \le k}{\vdash_{k'} G, W}\ \text{T-Sub} \qquad \frac{\vdash_k \mathsf{p}{\to}\mathsf{q} : brs, W \qquad brs' \subseteq brs}{\vdash_k \mathsf{p}{\to}\mathsf{q} : brs', W}\ \text{T-Narrow}$$

are admissible (✓ `tolerance_downward_closed`, ✓ `repair_narrow_sound`). T-Sub is subsumption in the budget index. T-Narrow looks like covariance of internal choice, the direction session subtyping already has, but it is not, and the difference is worth stating: the *condition* is closed under removing branches while *conformance is not* (✓ `narrowing_breaks_conformance`), because T-Comm asks the sender to offer exactly the protocol's label set. A session that still offers the removed label conforms to the wide protocol and not to the narrow one; narrowing both restores it (✓ `narrowing_asymmetry`). So narrowing is a rewrite of the contract, not a coercion along a subtyping preorder — which is the honest reading of what the gate does, since it refuses the removed label at run time and a process that still offers it has stopped describing what can happen. That is also why narrowing is a repair (§11) rather than a coincidence.

▶ **Theorem 11** (k^* is principal)**.** *k^* is not merely where a scan stopped. If a residual is k^*-tolerant and not $(k^*{+}1)$-tolerant, then the budgets at which it is tolerant are exactly those $\le k^*$ (✓ `principal_characterises`), such a k^* is unique (✓ `principal_unique`), and one exists whenever the residual is 0-tolerant, some budget is not tolerated, and the question is decidable (✓ `principal_exists`).*

So every derivation of the condition for that residual is a derivation at some $b \le k^*$, and the one at k^* subsumes the rest by T-Sub; and computing k^* by an increasing scan, which is what the tool does, is sound.

Reachability is also monotone in Γ, so for a *fixed* hazard granting a tool can only lower k^* (✓ `tolerance_antitone_in_ctx`) — the formal case for least privilege, and a refutation of our own prior expectation that more tools mean more recovery.

Not resource-aware typing. The budget resembles the potential of resource-aware session types [22, 23] and graded modalities [24]; the resemblance misleads. Potential is consumed by the program's own execution and exhaustion is a type error. The budget is consumed

by an *adversarial* choice the program does not make, so the rule quantifies over deviations rather than amortising cost, and exhaustion is unconstrained.

7 From Protocols to Programs

Everything so far concerns the global type's semantics. The theorem that makes it a discipline about *programs* couples the condition to the base judgment $G \vdash (\mathbb{M}; W)$, lifted to guarded choice without touching the guards: T-Comm still requires the sender to offer exactly the protocol's labels and every branch to check against the same residual; which label the participant eventually picks is its own business.

▶ **Definition 12** (Instrumented head-move semantics). *A session step carries the acting role and its cost:* 0 *for an action or an intended selection,* 1 *when the sender picks a label whose guard fails in the current world (Coq:* `hstep`*). A run records the trace of (role, cost) events;* total *sums costs and* percost_r *sums those of role* r*.*

▶ **Theorem 13** (Bridge). *If* $G \vdash (\mathbb{M}; W)$ *and* $\vdash_k G, W$*, then every instrumented head step of cost* $c \le$ *the remaining budget preserves typing and debits the budget by exactly* c *(✓* `bridge_step`*); consequently, along every run of total cost at most* k*, no configuration is a hazard state (✓* `bridge_every_configuration`*) — runs are prefix-closed (✓* `hrun_split`*), so this really is every configuration and not only the last (✓* `bridge_prefix`*).*

Two questions come before any of this, and a discipline that skips them earns the vacuity its predecessor died of (§13). Does a conforming session exist at all? And what does the fault model do to communication safety?

▶ **Theorem 14** (Non-vacuity). *Every two-role protocol is inhabited, over any runtime in which a tool call always has an answer. Read the global type as a pair of processes — canon: the sender offers exactly the protocol's labels, the receiver accepts exactly them, each role's actions become its own prefixes, and every other role is idle — and the result conforms in every world (✓* `canon_conforms`*), so the bridge applied to any such protocol is a statement about a session that exists (✓* `two_role_bridge_nonvacuous`*). The running example and its narrowed repair are inhabited in particular (✓* `Gbad_inhabited`*, ✓* `Ggood_inhabited`*), and that session really does take the misselected branch: the wrong label is a step it performs (✓* `MBad_takes_the_wrong_branch`*).*

The construction's side conditions are worth listing in full, since two of them are real and two are convenience. Real: distinct labels at a choice, because a label must name one continuation; and no goal markers, because CT-Goal demands φ at the world where the marker sits and no construction uniform in the world can supply that — Theorem 15 seen from the other side. Convenience: every choice is fixed to run from role 0 to role 1 and every action is performed by one of them, which is stronger than the proof needs. Beyond two roles, inhabitation is a projection question and this discipline has none: processes are typed directly against the global type. Projection would reappear only to *witness* inhabitation for bystander roles, under the classical plain-merge condition; the repairs' witnesses are exhibited individually. The goal marker is priced by the same rule. $\checkmark\varphi$ is an assertion, and CT-Goal is the only rule that reads one: the condition, the protocol steps, reachability and the swap relation all treat a marker as transparent. That asymmetry is deliberate, and Theorem 15 is where the premise is paid back.

The second question has a clean answer: nothing.

▶ **Theorem 15** (Markers are met). *If a conforming session of a b-tolerant protocol runs, within budget, to a residual whose head is ✓φ, then φ holds at the world it arrived in (✓ markers_are_met). Hence the residual is not Futile there: the goal is not merely reachable, it holds (✓ marker_reached_is_not_futile). This is the only formal link between the marker and the goal predicate Φ of §5, which is otherwise a separate parameter.*

The premise is not vacuous: the narrowed booking protocol with the goal marked where its safe path establishes it is tolerant at every budget (✓ `Ggoal_is_k_tolerant`), the two-role session conforms to it (✓ `Ggoal_inhabited`), and every run of that session that reaches the marker has achieved the goal (✓ `Ggoal_marker_met`).

The price deserves stating as plainly as the benefit, because the condition and conformance come apart on a marker. Put one on a branch a misselection can take: the condition does not see it — $\vdash_b$ treats a marker as transparent — while conformance demands φ at a world the protocol does not control. The result satisfies the condition at *every* budget and *no* session conforms to it (✓ `Gmiss_safe`, ✓ `Gmiss_uninhabited`). A marker is an assertion, so it belongs where the session can establish it and nowhere else, which is also why Theorem 14's construction excludes markers. Our benchmark carries the goal as a pack-level condition and uses no inline markers, so no measurement depends on this.

▶ **Theorem 16** (Progress under misselection). *A typed session that has not finished can always take a head step (✓ progress), and at a choice it can take one for* every *label the sender may pick, wrong ones included (✓ every_label_steps), because T-Comm makes the receiver offer at least the protocol's labels. A misselection is therefore never a communication mismatch, an unexpected label or a deadlock. Its only consequence is the world it leaves behind — which is exactly what §5 measures. (Assumption 5.)*

This is what separates the fault model from the crash-stop line [8, 9]: there a failure removes a participant and the metatheory must reconstruct progress; here progress is untouched, and the whole question is which world the session ends in.

▶ **Theorem 17** (Budgets distribute over participants). *Give each role r an allowance k_r. If the allowances of the roles that act sum to at most k, a session typed against a k-tolerant protocol is hazard-free on every run in which each role stays within its own allowance.* ✓ *budget_distributes*

Theorem 17 is the cross-participant composition result: the global budget is a contract that can be *split* among participants, so a deployment can hold each role to its own allowance. It is a statement about runs, not a per-role typing judgment — the discipline has no projection, so there is nothing to check per participant — and the mechanized content is that per-role costs sum to the total (✓ `total_eq_sum_percost`) and the bridge then applies.

Recursive sessions. The finite fragment is not the scope of the bridge. Section 8 gives recursive global types $\mu X.G$ their semantics by unfolding; processes carry μ likewise, and a process *unfolds to a head* (deterministically) before it is inspected, so a non-contractive process types against nothing. The conformance judgment becomes coinductive, with one added rule that unfolds the protocol's μ (Coq: `ctypes`, `hstep` in `Mu.v`).

▶ **Theorem 18** (Bridge, recursive). *An instrumented head step of a typed session is a protocol path of the same misselection cost, and typing is preserved (✓ sim_step). Hence a session typed against a recursive protocol satisfying the condition at budget k (Theorem 26) is hazard-free on every run of cost at most k (✓ bridge_mu_safeR, semantically ✓ bridge_mu). On the finite fragment this recovers Theorem 13 through Theorem 25 (✓ bridge_finite_via_mu).*

Bystanders. The head-move semantics is the gate's: an out-of-turn attempt is refused. An ungated deployment lets a *bystander* — a role the head does not involve — fire its next capability early. Since the protocol is a schedule of world effects, that is not automatically harmless: it is exactly what the reorder repair exploits. We give the permutation relation and the conditions under which it is (Coq: `Interleave.v`). A *swap* exchanges two adjacent independent nodes, in either direction and under any context:

$$
\begin{array}{ll}
b@\mathsf{q}.\, a@\mathsf{r}.\, G \rightleftarrows a@\mathsf{r}.\, b@\mathsf{q}.\, G & \mathsf{q} \neq \mathsf{r},\ a, b \text{ independent} \\
\mathsf{p}{\to}\mathsf{q}\{\ell_i[\psi_i].\, a@\mathsf{r}.\, G_i\} \rightleftarrows a@\mathsf{r}.\, \mathsf{p}{\to}\mathsf{q}\{\ell_i[\psi_i].\, G_i\} & \mathsf{r} \notin \{\mathsf{p}, \mathsf{q}\},\ a \text{ preserves each } \psi_i \\
\checkmark\varphi.\, a@\mathsf{r}.\, G \rightleftarrows a@\mathsf{r}.\, \checkmark\varphi.\, G & a \text{ preserves } \varphi \\
\mathsf{p}{\to}\mathsf{q}\{\ell_i.\, \mathsf{r}{\to}\mathsf{s}\{m_j.\, K_{ij}\}\} \rightleftarrows \mathsf{r}{\to}\mathsf{s}\{m_j.\, \mathsf{p}{\to}\mathsf{q}\{\ell_i.\, K_{ij}\}\} & \{\mathsf{p}, \mathsf{q}\} \cap \{\mathsf{r}, \mathsf{s}\} = \emptyset
\end{array}
$$

Communications change no world, so their permutation needs no condition on guards or effects — but it does need the two choices to be genuinely independent, which the rule expresses by requiring the full product: every pair (ℓ_i, m_j) present, with a residual K_{ij} that depends on the pair and not on the order. An action's permutation is conditional: a must be *hazard-neutral* (never change whether the world is a hazard), a and b must *commute* (one order re-serializes as the other with the same end world) and preserve each other's enabledness, and a must preserve the guards and goal markers it passes.

▶ **Theorem 19** (Bystanders). *Safe swaps preserve the condition at every budget (✓ `swap_safe`) and preserve typing — subject reduction under permutation (✓ `swap_ctypes`). Hence a typed session of a k-tolerant protocol is hazard-free on every* interleaved *run of cost at most k, where before each step the protocol may be permuted by any sequence of safe swaps (✓ `bridge_interleaved`).*

▶ **Theorem 20** (Variable disjointness suffices). *For STRIPS capabilities stated extensionally — a precondition supported on a variable set, add and delete lists — if the effects of each action are disjoint from the other's footprint (its support and effects), the actions commute and preserve each other's enabledness (✓ `strips_commute`, ✓ `strips_enables`); an action whose effects are disjoint from the support of a predicate preserves it (✓ `strips_preserves`, ✓ `strips_neutral`). Together these discharge every side condition of an action swap (✓ `strips_swap_act`).*

Theorem 20 is what the tool checks: for every capability action and every earlier node of another role it could pass, it compares footprints — taking the goal's and every precondition's atoms as the support of the derived hazard and of the rational guards — and reports either that the head order is exact or the pairs the gate must serialize. The discipline is synchronous by design (the gate delivers a label when it is taken); what the theorem does not cover is asynchronous buffering, where a sent label waits in a queue, which changes the semantics rather than merely permuting it.

8 Regular Protocols

A regular global type unfolds to a finite graph of protocol nodes; over the finite abstract world of the widened fragment, budgeted reachability is reachability in the product $\mathit{Node} \times \mathit{World} \times \{0..k\}$. The development has two layers: an abstract one over any node and world types with computable successors (Coq: `Regular.v`), and its instantiation to μ-types, where the finiteness of the node set is a theorem rather than an assumption (`Mu.v`).

▶ **Theorem 21** (Product). $(n, w) \rightsquigarrow_b^{\mathcal{H}}$ *iff a hazard state is reachable from (n, w, b) in the product graph, whose misselection edges decrement the third component (✓ `product_correspondence`).*

▶ **Theorem 22** (Loops meet the budget). *The budget component never increases along a product path (✓ `budget_never_increases`); a path with d misselection edges from budget b has $d \le b$, and exactly $b - b'$ if it ends at budget b' (✓ `dev_edges_bounded`, ✓ `dev_edges_exact`). A loop containing a misselectable branch is therefore traversed wrongly at most k times within budget k.*

▶ **Theorem 23** (Decidability). *For a finite world type and a finite, successor-closed list of nodes with computable successors, the boolean `decide_reachb` decides $(n, w) \rightsquigarrow_k^{\mathcal{H}}$ from any listed node: it is sound (✓ `decide_sound`) and complete (✓ `decide_complete`), the latter via a pigeonhole argument that reachability is witnessed by a path no longer than the reachable-state count (✓ `reach_bounded_path`); together, ✓ `decide_reachb_correct`.*

From μ-types to the graph. Regular global types carry $\mu X.G$ and X (de Bruijn indices; Coq: `Gr`). Unfolding is substitution of the closed μ-term for its variable, and the protocol steps of §4 gain one silent step, $\mu X.G \to G[\mu X.G/X]$. Budgeted reachability on μ-types *is* the product of Theorem 21 over these steps (`reach_mu`).

▶ **Theorem 24** (The unfolding closure is finite). *For a closed regular type G_0 there is a computable list* $\mathrm{cands}(G_0)$ *— the closures of G_0's subterms under their enclosing binders — that contains G_0 and is closed under every protocol step (✓ `cands_closed`); every state reachable from G_0 in the product lies in it (✓ `reach_in_cands`). The heart is a substitution lemma: substituting the closed μ-term for the variable it binds, after closing the outer context, is closing the extended context (✓ `subst_comp`, ✓ `unfold_close`).*

▶ **Theorem 25** (Embedding). *The finite fragment embeds: $\rightsquigarrow_b^{\mathcal{H}}$ of §4 coincides with `reach_mu` on the image (✓ `reach_embed`), so T-Choice-Safe is exactly non-reachability of a hazard in the product graph of the embedded protocol (✓ `TC_regular`).*

Read T-Choice-Safe coinductively — a recursive protocol has no finite derivation, while a reached hazard still has a finite path. The rule is one clause, over the protocol's own steps rather than its syntax, with a double line for the coinductive reading:

$$\frac{\neg\mathcal{H}(w) \qquad \forall\, G \to_0 G',\ \vdash_b^{\nu} G', w' \qquad \forall\, G \to_1 G',\ b = b'+1 \Rightarrow \vdash_{b'}^{\nu} G', w'}{\vdash_b^{\nu} G, w}\ \text{T-Safe-}\nu$$

$$\vdots$$

where $\to_0$ is a compliant protocol step and $\to_1$ a misselection.

▶ **Theorem 26** (The condition, for recursive protocols). *$\vdash_k^{\nu}$ is again exactly non-reachability: sound (✓ `TR_sound`), complete (✓ `TR_complete`), together ✓ `TR_exact`. It is a conservative extension: on the finite fragment the two judgments coincide under the embedding (✓ `TC_is_TR`), and `decide_mu` decides it (✓ `decide_mu_judgment`). So the bridge, the decision procedure and the condition are statements about one object at both layers, not three.*

▶ **Theorem 27** (Decidability for μ-types). *Over a finite abstract world with decidable guards and computable tool successors, `decide_mu` decides $(G_0, w) \rightsquigarrow_k^{\mathcal{H}}$ for every closed regular G_0: it runs Theorem 23's procedure on* $\mathrm{cands}(G_0)$*, which Theorem 24 shows is successor-closed (✓ `decide_mu_correct`).*

▶ **Theorem 28** (Guardedness, and where it is needed). *Conformance for recursive protocols is coinductive, so $\mu X.X$ is typable against every session and world (✓ `ctypes_mu_var`) — and it can never take a step (✓ `unguarded_var_stuck`). Progress therefore fails without a guardedness*

condition, and this is the only place in the development that needs one. Call a type guarded *when no recursion variable occurs at the head of its own binder after peeling μs and goal markers (*`gd`*); markers do not guard, because a step erases one rather than firing on it, and $\mu X.\checkmark\varphi.X$ is stuck for the same reason (✓* `unguarded_goal_stuck`*). Guarded unfolding leaves the leading prefix no longer than it was (✓* `pre_unfold`*) and preserves guardedness and closedness (✓* `gd_unfold`*, ✓* `closed_unfold`*), so a closed guarded conforming protocol that has not finished can always step (✓* `progress_mu`*). Dropping* `gd` *makes that false, witnessed by $\mu X.X$ (✓* `contractiveness_is_necessary`*).*

Theorem 27 carries no guardedness hypothesis, and that is not an oversight: the candidate set is finite by construction (Theorem 24) rather than by contractiveness, so the decision procedure — the part the tool runs — terminates on non-contractive input too, and answers correctly about it.

▶ **Proposition 29** (Tolerance degree). *k^* is computable. Whether $k^* = \infty$ is plain reachability in Node $\times$ World with misselection edges made free; otherwise k^* is at most the number of misselection edges on a shortest hazard path, bounded by the product size, and is found by iterating Theorem 23. With $|World| = 2^{|\mathsf{Pred}|}$ the decision problem is in PSPACE; each iteration is linear in the product.*

Proof. Paper proof from Theorems 21–23: dropping the budget component yields the ∞ case; `dev_edges_exact` bounds k^*; reachability in a succinctly presented graph is in PSPACE. ◀

9 Why a Discipline, and Not a Model Checker

Theorem 10 invites the objection that the rules are a decision procedure in disguise; Naik and Palsberg [20] gave the standard reply, and we rest instead on a theorem and an experiment.

▶ **Theorem 30** (Modular composition). *If $\vdash_k G_1, W$ and $\vdash_{b'} G_2, W'$ for every (b', W') at which G_1 can end from budget k, then $\vdash_k G_1; G_2, W$ (✓* `TC_seq`*). In interface form: for any I containing those exit points, it suffices that $\vdash_{b'} G_2, W'$ for all $(b', W') \in I$ (✓* `TC_seq_interface`*). Sequencing is defined on the finite fragment only: $\vdash_k^\nu$ has no composition theorem, so modular checking does not currently reach recursive protocols.*

The theorem is what a whole-system reachability run cannot be: k becomes a contract on a boundary. It does not say that boundary is small, and the evaluation shows it is not — the *concrete* exit set grows exponentially when every segment leaves its own atoms behind. What makes modular checking pay is the interface form with a coarser I, and the principled coarsening is the cone of influence.

▶ **Theorem 31** (Cone of influence). *Let V be a set of state variables such that the hazard reads only V, every guard of G reads only V, and every capability G* invokes *maps V-agreeing worlds to V-agreeing successors — the last restricted to G's own tools, not to all of Γ, because a modular interface is checked against a residual and the wider hypothesis is one the construction that uses it would refute. Then worlds agreeing on V are interchangeable for G at every budget: hazard reachability transports (✓* `reach_cone`*) and so does the condition (✓* `safeT_cone`*). Consequently an interface need only contain one representative of each V-class, not one world per concrete exit (✓* `interface_projection`*). For* STRIPS *capabilities that last condition is a containment the tool can check — it suffices that every* precondition *of an invoked tool reads only V (✓* `strips_cap_cone`*, ✓* `strips_safeT_cone`*) — and what an action* writes *outside V does not matter, since worlds are compared only on V.*

That abstraction is what makes computing an interface cheap rather than exponential (Table 2). What the theorem does not do is compute V: the tool takes the cone syntactically from the predicates the remaining segments mention, and that this syntactic cone satisfies the containment is a property of the front end. This is the axis on which MPST is currently being argued [17]; the community has accepted parameterising typing by a model-checked property [8, 9], and the theorem plus the projection is our concrete instance of that move.

10 Implementation and Evaluation

`skillc severity` implements the analysis over the existing achievability compiler: the checker's own symbolic search (predicates concrete, numerics symbolic, widened at loop heads) is run pointwise at every branch of every choice, at every abstract world and budget up to $k_{\max} = 4$, with the default guards and hazard of §4–§5 and explicit overrides where declared. It reports per-branch severity, k^*, the first hazard witness, the point-of-no-return action, the narrowing repair, and the bystander pairs the gate must serialize. Nineteen tests check the tool's verdicts on the paper's instances against the Coq development's, including two that check the tool where a theorem licenses a shortcut: that k^* really is a threshold on every benchmark protocol (Theorem 11), and that projecting an interface onto the cone of influence never changes what the modular check concludes (Theorem 31).

Verified kernel. The tool's search is the graph form of Theorem 24, with widening at loop heads over the arithmetic state; it is not extracted from the proof. To close that gap on the fragment where the proof applies we built a kernel (Coq: `Kernel.v`) that instantiates `decide_mu` on bit-vector worlds with STRIPS actions, guards as world sets, and the derived hazard as a bit that an irreversible action sets whenever the goal is no longer reachable from its continuation. The decision — the least budget at which a hazard is reachable — is proved correct (✓ `kernel_first_spec`); goal reachability, used to derive rational guards and hazard bits, is proved correct (✓ `goal_reachable_correct`); the decision procedure itself was replaced by a deduplicating, early-exiting variant proved sound and complete (✓ `decide'_sound`, ✓ `decide'_complete`).

The kernel's goal notion is not the theory's, and the two are *incomparable* rather than one being safely stronger. It requires the protocol to run to completion, where Φ-reachability fires at any Φ-world (✓ `goal_reach_strictly_stronger`), and it spends no budget, where Φ-reachability is budgeted (✓ `goal_reach_ignores_the_budget`); only the first direction has an implication, and it reads the budget off the witness path (✓ `goal_reach_implies_reach_mu`). This matters beyond the severity labels: the elaboration derives the *rational default guard* and the *derived hazard bit* from the same notion, so it is a front-end convention the theory does not fix, and k^* is computed against it. Both the analyzer and the kernel implement that convention, independently; what the agreement below establishes is that they agree, not that the convention is the paper's Φ-reachability. The elaboration from the tool's protocol tree into the kernel's input is the trusted front end: a hundred lines of executable Coq and a hundred of Python. The kernel is extracted to OCaml and run by `skillc severity -verified`. On the sixteen benchmark protocols in its fragment — propositional effects, no environment choice — it agrees with the analyzer's tolerance degree on every one, in under 50 ms each.

Sixteen hand-picked protocols are a weak test of that agreement, so we also compare the two on random ones (`scripts/differential_test.py`). The generator samples packs inside the shared fragment and is otherwise unconstrained: nested choices, explicit and

rational guards, named recursion, irreversible tools, unreachable goals. The analyzer and the extracted kernel share no code, so a disagreement is a defect in one of them. Across two seeds (150 at `20260902` and 350 at `4242`, both at $k_{\max} = 2$), 500 protocols were generated, 500 compared, and the two agree on the tolerance degree 500 times and disagree none. An earlier campaign at the same seeds compared 499: one generated pack carried two recursions with the same name and was rejected by the pack schema, a generator bug rather than a tool one, since fixed. Two thirds of the sampled protocols have no reachable hazard at all, so most agreements are agreements that nothing is there; the discriminating cases are the third that carry a finite tolerance degree, and $k_{\max} = 2$ means no degree above two is ever distinguished.

This is the strongest evidence we can offer that two independent implementations — a hand-written symbolic analyzer and an extraction of the mechanized decision procedure, sharing no code — compute the same tolerance degree, and it is a test the discipline makes possible: without a decision procedure proved correct there is nothing to differ from. What it does not establish is that the goal convention both of them use (above) is the Φ-reachability of §5; that gap is a front-end one and we leave it open.

Finding 1: existing corpora cannot pose the question. Over the compiler's achievability corpus (15 packs), its extension (6) and the 17 public skills of the reference collection, *no* pack declares an irreversible effect and the real skills contain *no choice point at all*: all 38 are trivially tolerant at every k. That is a result about modelling. Real-world irreversibility — sending, paying, deleting, deploying — is not captured by achievability-oriented compaction, so severity needs the declaration §5 introduces, and the question this paper asks has been invisible to the corpora the field evaluates on.

Finding 2: the severity benchmark. We therefore built seventeen protocols with genuine decision points and irreversible tools, each with a provenance note stating the expected verdict *before* running the tool: booking (the running example and its two repairs), database migration, an email campaign and its guard repair, deployment with and without rollback, file cleanup, order fulfilment with an environment-controlled bank, a retry loop with a destructive give-up, a guarded shipping choice, an insurance claim in two worlds, a staged commit, and two release protocols with a bystander role — one whose bystander is variable-disjoint, one whose bystander establishes a later precondition. Results (Table 1): 55 branch verdicts (a branch at an abstract world) in 0.21 s total; 29 Benign, 7 Futile, 19 Catastrophic. That count is over *all* branches, and the reader should not take the 29 as the Benign class of the trichotomy: 28 of them are branches whose guard *holds*, so they are not misselections at all. Restricted to misselections the split is 1 Benign, 7 Futile, 19 Catastrophic— and the single Benign misselection is the express detour of `shipping_detour`, whose guard is explicit. The class the slogan needs is the one this benchmark is thinnest on, and Finding 4 tests it with $n = 1$.

Which results the benchmark exercises. A choice is a communication $\mathsf{p}\to\mathsf{q}$ with $\mathsf{p} \neq \mathsf{q}$, so a protocol whose choices name no recipient has no conforming session — CT-Comm wants the roles to differ. Six protocols have a second role and their eight choices name it; the other eleven model a lone agent, with nobody to tell. The condition, k^*, the severity classes and the repairs are about the protocol and apply to all seventeen; the bridge, progress, budget distribution and the bystander results are about sessions and apply to five of the six — `order_fulfilment` is out twice over, since two of its choices are the environment's and its choices run in both directions between one pair. That is a real limit of this benchmark rather

protocol	guards	choices	loops	irreversible	k^*	Benign	Futile	Catastrophic
booking_fastpath	default	1	0	purchase	0	1	0	1
booking_reordered (repair: hoist)	default	1	0	purchase	≥ 5	2	0	0
booking_narrowed (repair: narrow)	default	1	0	purchase	≥ 5	1	0	0
migration_backup	default	2	0	drop_old	1	3	1	2
email_campaign	declared	1	0	send	0	1	0	1
email_campaign_guarded (repair: hoist)	declared	1	0	send	≥ 5	2	0	0
deploy_with_rollback	default	1	0	—	≥ 5	1	1	0
deploy_no_rollback	default	1	0	deploy	0	1	0	1
file_cleanup	default	1	0	delete	0	1	0	1
order_fulfilment (environment choice)	declared	3	0	ship	0	1	0	1
retry_then_purge	default	2	1	purge	0	6	0	2
shipping_detour (explicit guards)	explicit	1	0	—	≥ 5	2	0	0
claim_eligible	explicit	1	0	refund	≥ 5	1	1	0
claim_ineligible (same protocol)	explicit	1	0	refund	0	1	0	1
staged_commit	default	3	0	commit	1	3	4	7
release_with_audit (bystander, exact)	default	1	0	deploy	0	1	0	1
release_with_cache (bystander, serialize)	default	1	0	deploy	0	1	0	1

Table 1 The severity benchmark ($k_{\max} = 4$; ≥ 5 means no hazard within four misselections). Verdict counts are per branch and abstract world, so 29 Benign counts 28 branches whose guard holds; restricted to misselections the split is 1/7/19. Total analysis time 0.21 s.

than of the theory, and the two release protocols were built to exercise the bystander half. k^* distribution 9×0, 2×1, $6 \times \geq 5$; the nine distinct point-of-no-return actions it names across all seventeen are `purchase, send, deploy, delete, ship, purge, refund, drop_old, commit` — every one an externally visible effect. Sixteen of seventeen verdicts matched the pre-stated expectation; the exception exposed an authoring error in our benchmark (a goal disjunct that failed to exclude `shipped`), corrected and recorded. The bystander check (Theorem 20) reports the audit release exact — five independent pairs — and the cache release with two pairs to serialize, both on the atom the bystander establishes; every other benchmark protocol has a single acting role, so the question does not arise there.

Which repairs the benchmark exercises. Two of its protocols are repairs of others, and both apply the same transformation: *hoisting* an action performed in every branch to above the choice, so that the wrong selection can no longer skip it. Hoisting is the bystander swap of Theorem 19 when the actor is uninvolved in the choice (✓ `SW_comm`, ✓ `SW_comm_rev`); in both instances the actor *is* one of the communicating pair, so no theorem of ours licenses it and what the table reports is the tool's verdict on the result, not a certified rewrite. The guard and reorder repairs *as mechanized* are exercised in Coq (§11) and not here, because the pack language has no abort world to divert to; closing that gap is future work.

Three pairs carry the paper's claims. *Repairs restore tolerance:* narrowing, and hoisting the establishing action above the choice, each take a 0-tolerant protocol to tolerance at every k. *Severity is a property of node and world:* the insurance-claim protocol is identical in both rows; with an eligible claimant the wrong branch is Futile, with an ineligible one `refund` is a point of no return and $k^* = 0$. *The point of no return depends on Γ's establishers:* with `rollback` available, `deploy` deletes nothing permanently and the wrong branch is merely Futile; remove `rollback` and the same branch is Catastrophic. The last pair refines ✓ `tolerance_antitone_in_ctx`'s anti-monotonicity (§6): for a *fixed* hazard more tools never help, but the derived hazard is not fixed — a tool that re-establishes a lost atom removes a point of no return.

Finding 3: modularity pays only with the interface abstraction. We chained a benchmark segment n times, $G; G; \cdots; G$ with atoms renamed per segment, and compared four analyses

n	whole, goal queries	exits	modular concrete, queries	interface	modular projected, queries	interface
1	8	3	8	3	8	2
2	26	6	29	8	16	2
3	62	12	81	20	24	2
4	134	24	205	48	32	2
5	278	48	493	112	40	2
6	566	96	1149	256	48	2

Table 2 Chains of the migration segment, $k = 2$. Re-check after replacing the last segment: whole-system 15–109 ms growing with n; modular 12–16 ms, constant. The deployment segment gives the same shape at smaller scale (470 ms vs 40 ms at $n = 6$).

at $k = 2$ (Table 2): the whole-system *complete* enumeration of hazard-free terminations (like-for-like with computing an interface), modular checking with the *concrete* exit set, modular checking with exits *projected* onto the cone of influence of the remaining segments, and the cost of re-checking after the last segment is replaced by its unsafe variant.

State first what modularity does *not* buy, because the natural comparison is the wrong one. Deciding whether a hazard is reachable in the whole chain is already linear for the tool: on the migration segment it asks 7 goal-reachability queries at $n = 1$ and 22 at $n = 6$, fewer than the projected modular analysis' 48. Nobody needs to decompose that. What is exponential is computing an *interface* — enumerating the hazard-free terminations a segment leaves behind — and that is the like-for-like comparison, because it is what a boundary contract costs. There the complete whole-system enumeration asks 566 queries at $n = 6$ against 48 (22.4 s against 0.083 s; we report the counters, which reproduce exactly). The concrete interface asks 1149, more than the whole system, because it grows exponentially (256 points at $n = 6$) — computing an unabstracted interface is worse than not abstracting at all, and the projected one has two points at every n.

The win that survives is the practical one: re-checking after a change costs whole-system analysis the full run (22 queries, 109 ms at $n = 6$, growing with n) and modular analysis the last segment alone (8 queries, 13 ms, constant in n).

Composing through an interface rather than internals is a line of work in MPST itself [50, 51]; what the budget adds is that the interface carries a *quantity* — how much tolerance is left — not only a state. Theorem 30 does not make checking cheap; it makes it *decomposable*, and the interface form is what lets a coarse invariant stand in for the concrete exits. Inferring good interfaces beyond the cone of influence is the open engineering problem.

Finding 4: live agents take the branches the tool calls catastrophic. Predictive validity asks whether the Catastrophic verdicts name choices real agents actually make. We drove every benchmark protocol with two production language models of one family, a small and a mid-size one, through their vendor's command-line interface with no tools and a two-sentence system prompt, each playing every agent role. At a choice the model sees the goal, the facts now true, the tools with their effects, and each option's actions and continuation, and answers with a label; everything else — effects, preconditions, loops, the environment's choices (sampled uniformly), the hazard — is the analyzer's own semantics. Two conditions: *plain*, the task as stated, and *pressured*, the same plus a note claiming the preparatory steps were done and asking for the fastest route. Five runs per protocol, model and condition: 340 runs, 400 agent decisions, $1.59 (`scripts/live_agents.py`; every run is in `results/live_agents_runs.jsonl`). The harness does not record the model identifier the vendor actually served, so we claim no reproducibility against a moving endpoint.

Two checks of the theory hold exactly. Every catastrophe had more than k^* misselections and none occurred on a protocol tolerant at every tested budget — the harness and the tool share semantics, so this checks the harness against Theorem 13. And the verdicts are not vacuous: 6 of the 19 Catastrophic verdicts were taken by a live agent (17 times), against 1 of the 7 Futile ones (once); the one Benign misselection — the express detour, intended only when urgent — was taken in all ten pressured runs and delivered every time, which is what Benign means.

No rate claim is supported, and we withdraw the one we made. The catastrophe counts look ordered — 0 of 120 runs on $k^* \geq 5$ protocols against 7 of 220 on $k^* \in \{0, 1\}$ — but the first number is forced. Each of those six protocols has a single choice point, so a run can contain at most one misselection, and each tolerates at least four: a catastrophe there is arithmetically impossible, and the harness's own scripted chooser, which picks wrongly one time in three and consults no model, reproduces 0/120. Inside the low-k^* group the counts *invert*: 2/180 at $k^* = 0$ against 5/40 at $k^* = 1$ (Fisher $p = 0.0025$). Nor does the misselection frequency say anything: 11/120 on the tolerant protocols against 2/180 at $k^* = 0$ looks like agents erring where it is safe, but 10 of those 11 are the single Benign detour, whose guard is `urgent` and whose pressure prompt asserts exactly that — an agent taking it is arguably reading the guard correctly, not ignoring risk. Misselection in this sample is concentrated in two branches under one condition, which is enough to show the verdicts name real choices and not enough to say anything about rates. The repair pairs behave as expected: the two catastrophes of `booking_fastpath` under pressure vanish in its hoisted and narrowed forms with the same model and prompt — the narrowing is Theorem 32, the hoist is the transformation the previous paragraph says no theorem of ours licenses.

Misselection was rare and concentrated. Neither model misselected once in the 170 plain runs; under pressure the smaller model skipped the preparation stages of `staged_commit` and committed in 5 of 5 runs, and took the booking fast path in 2 of 5, while the larger misselected only on the benign detour; the two bystander protocols drew none in 40 runs. Two disclosures. An earlier prompt that hid what the environment's branches do led the larger model to ship before charging in 5 of 5 plain runs; we count that against the prompt and report the re-run with the branch contents shown, but the discarded runs were not retained, so a reader cannot audit that revision — a fault in our record keeping. And the experiment exposed a reporting bug: the recording pass stopped at the first hazardous branch, so a sibling could go unclassified. It was fixed before the numbers above and Table 1 reflects the fix.

Finding 5: on real skills, the checker's refusals are the runs that would have been wasted or wrong. The achievability half of the discipline is only useful if its verdicts predict what happens when an agent actually runs a skill. We took real skills from public repositories — the seventeen of the vendor's skills repository and 145 more from twelve third-party collections, de-duplicated by content hash and each recorded with its source, fetch date, licence note, SHA-256 and byte count, all 162 of which verify against the shipped files — and two runtimes: one with a shell and file tools, one with file tools only. A code fence the document expects the agent to execute is an invocation of the shell (a front-end rule added for this experiment), so a script-based skill is certified in the first runtime and refuted with MissingCapability in the second. Over the 162 skills, 149 are certified in their home runtime and 108 refuted in the file-only one, 95 flipping between them; 13 are refuted even at home. We audited every one of those by hand (`benchmarks/home_refutation_audit.json`): six are genuine — the document tells the agent to *call* a tool no runtime in our set provides, in every case

from an MCP server — and seven are *misextractions*, where the deterministic reader took a code identifier for a tool (a dataframe method, settings keys, a crate, a meta-tag name, a command-line program that in fact acts through the shell). As a rate over the corpus that is a false refutation on 4.3% of 162 skills; as precision of the home refutations themselves it is 6 of 13, 46%, and that is the number to carry. Refutations are sound *for the pack the front end produced*, and a misextraction produces a different pack. We have not audited the other direction — 149 home certifications and 108 file-only refutations are unchecked — so no false-*certification* rate is claimed, and the audit is single-rater and ours. The census takes 2.1 s and no tokens.

We authored nine specification pairs whose B variant makes a claim the tools cannot honour — three bounds the only tool cannot meet, three guards nothing satisfies, two goal conditions with no establisher, one undeclared tool (`benchmarks/spec-cases`). For eight skills with a task the sandbox can carry out (the rest need a package, a GPU or a credential) and for four of those nine pairs — the five added after the agent runs were checked statically only — we ran the same two models as in Finding 4 in an empty directory with the skill and a concrete task, tools restricted to the runtime, and *verified the artifact*: a workbook's live formula, a merged PDF's page count, the order a mock tool recorded. A run that claims completion and fails verification is a *silent wrong result*; one that says it failed is an *honest failure*; either way its tokens are spent.

Table 3 summarizes 134 runs. On the runs the checker certified, 32 of 32 real-skill runs and 16 of 16 specification runs produced a verified artifact, with no silent wrong result. On the runs it refuted, the outcome depends on one thing: whether the skill's procedure is the only route to the artifact.

Where it is, no refuted run succeeded: the three document skills produce binary formats, and all 12 file-only runs failed honestly. The four specification B variants make a claim their tools cannot honour, and 12 of 16 runs failed honestly, 2 produced a silent wrong result — the small model fabricated the approval no tool could give and reported the report published — and 2 ended without the status line the harness asks for. Those 28 runs cost $1.09 and 1.56 M tokens.

Aggregated over every refuted configuration the picture is not one-sided: 20 of the 66 refuted runs did produce a verified artifact, and all 20 are the small-input cases discussed next. The partition below is post hoc, and we report the aggregate first so it is not hidden by it.

Where it is not, the first version of this experiment refuted five third-party skills whose tasks reduced to arithmetic over a dozen rows, seven boolean checks or a frontmatter grammar, and the file-only agent ignored the skill's scripts and computed the artifact by hand in 20 of 20 runs ($1.29, 1.22 M tokens). That was a measurement of our task sizes, not of the checker. We therefore rebuilt those five tasks at realistic scale — a thousand transcripts across four samples, a thousand-row employee table, five hundred user records, a hundred and fifty manifests, a hundred and twenty draft files — with verifiers that recompute the answer from the generated inputs and compare it exactly, so an estimate cannot pass. The checker's verdicts are unchanged by the scale-up. The agents' behaviour is not: in the certified runtime 19 of 20 runs still produced a verified artifact ($1.65, 3.19 M tokens, median 7.5 turns), while in the refuted one 0 of 18 did: 6 honest failures, 5 silent wrong results and 7 without the status line the harness asks for, at a median of 20.5 turns, $13.21 and 14.28 M tokens — 4.5 times the tokens of the certified runtime for no artifact at all. The median understates the tail: six of the 18 refuted runs ran 65 to 127 turns before giving up or getting it wrong, against a maximum of 13 in the certified runtime. Hand computation is a way past the

configurations	checker	runs	verified	silent wrong	honest fail	no verdict	agent cost	agent tokens
8 real skills, shell runtime	certified	32	32	0	0	0	$1.83	3.03 M
3 document skills, file-only	refuted	12	0	0	12	0	$0.62	0.49 M
4 specification pairs, A	certified	16	16	0	0	0	$0.50	1.14 M
4 specification pairs, B	refuted	16	0	2	12	2	$0.47	1.07 M
5 skills, small inputs, file-only	refuted	20	20 (by hand)	0	0	0	$1.29	1.22 M
5 skills, realistic inputs, shell	certified	20	19	1	0	0	$1.65	3.19 M
5 skills, realistic inputs, file-only	refuted	18	0	5	6	7	$13.21	14.28 M

Table 3 Real skills and specification cases, two runtimes, two models, two runs each, except the realistic-scale file-only row, which has 18 runs rather than 20 because one cell recorded two; every artifact verified by recomputation. "Verified" counts artifacts that pass the verifier whether or not the status line was reached. The last three rows are the same five skills at two input sizes. The checker's total time over all 34 configurations is 228 ms.

checker only while the input is small enough to hold in a context window; the verdict was right and the earlier task was too easy.

How much of this a regular expression already explains. The honest objection to the corpus half of this experiment is that most of the checker's refutations are a capability set-difference: the document contains a shell-looking fence and the runtime has no shell. We implemented exactly that baseline (`scripts/grep_baseline.py`) and scored it on 34 configurations whose answer is known independently of the checker — eighteen specification variants, fixed by construction, and sixteen runtime configurations the live runs settled by producing or failing to produce a verified artifact (ten of them at realistic input size, six at the smaller one). The grep gets 25, the checker 34.

The interesting part is *which* 25. On this dataset the grep's verdict is `runtime` $\neq$ `file-only` on every one of the 34 rows, so it is verdict-identical to a classifier that reads nothing but the runtime name and never opens the document; and on the sixteen observed rows the runtime alone settles the truth (eight shell, all achievable; eight file-only, none), so those rows separate nothing. Both baselines therefore score 16/16 there and 9/18 on the specification variants — right on every A variant, wrong on every B variant — while the checker gets 18/18. The nine B variants are the whole of the discriminating evidence, and each needs reachability rather than a set-difference: a bound the only tool cannot meet, a goal condition nothing establishes, a guard nothing satisfies. That is the honest division of credit, and it is narrower than the headline: most of the corpus census is a set-difference anyone could compute, the sixteen observed rows are the outcomes Table 3 already reports, and the specification cases — which are ours, and five of which were authored after the agent runs and are checked statically only — are what the analysis is for. Timing is not the comparison: the grep takes 0.2 ms over the 34 configurations and the checker 228 ms.

The corpus is an untrusted input. A skill document is not data: it is a set of instructions an agent will follow with real tools, so a corpus fetched from a dozen third-party repositories is an attack surface for the experiment itself. We scanned all 162 documents statically, before running anything, for six families of evidence — harness impersonation and instruction override, credential exfiltration, destructive commands, fetch-and-execute, obfuscation, and edits to the agent's own configuration (`scripts/scan_skills.py`, `docs/CORPUS_SECURITY.md`). No malicious skill and no injection payload aimed at an agent was found: zero hits for fake harness turns, hidden instructions in comments and invisible characters. Nine files raised a flag: eight benign in context, and one real but low — a skill that installs a vendor CLI by

piping a download into a shell, to the vendor's own domain. Another appends an API key to a shell profile; neither is among the sixteen we executed. A regression test fails the suite if a corpus refresh introduces an unreviewed hit. The one impersonation attempt we did encounter arrived in a *web search result* during collection, not in a skill file, which locates the real surface for an agent that browses.

Finding 6: what the check costs in tokens, and when it needs a model at all. The deterministic front end spends no model tokens; the question is when it is enough. The reading it falls back to — one act per invoked tool — *under*-approximates the document, so a refutation on it is sound for that pack whatever the document means, while a certification says only that the named tools exist; the 4.3% false-refutation rate above is exactly the residue. That asymmetry is a decision procedure for spending money: escalate to LLM compaction exactly when the pack is weak, the verdict is a certification, and the document visibly carries meaning the reader did not capture. `skillc autocheck` implements it, escalating on 130 of the 162 skills in the home runtime, where almost everything is weakly certified, and on 49 in the file-only one, where 108 are refuted for free. We attempted the compaction on 24 skills and 20 returned a usable pack; the other four produced no verdict and are counted as failures, not dropped. Over the 20, one compaction costs a median 22 440 tokens and $0.088 — 27.9% of the measured median agent run, paid once per skill version and amortized over every run of it, and zero for the 32 skills that need no model. The refuted runs that produced nothing — 46 of the 66, the other 20 being the small-input cases the agent computed by hand — spent 15.8 M tokens and $14.296 before failing. The check is cheap in the cases that matter because those cases are refutations, and refutations are the free half.

11 Repairs

When a branch is Catastrophic at the intended budget, four transformations make the mistake affordable: *guard* (insert a validation before the irreversible branch), *compensate* (add a recovery path), *narrow* (remove the branch, so the gate refuses it), and *reorder* (move the irreversible action after the validation). All have ancestors: amending asserted choreographies [35], interactional exceptions [36], supervisory control and shields [37, 38, 39], choreography repair [52, 53]. All four are mechanized (Coq: `Repairs.v`). A *validation* of ψ is a capability that fires exactly when ψ holds and changes nothing — a tool with precondition ψ and no effects.

▶ **Theorem 32** (Narrow). *Removing branches from a choice node preserves the condition at the same budget (✓ `repair_narrow_sound`).*

▶ **Theorem 33** (Congruence, and the repairs off the root). *The condition is a congruence for one-hole contexts: if G_2 is b-safe wherever G_1 is, then $C[G_2]$ is b-safe wherever $C[G_1]$ is, for a hole under actions, markers and any branch of any choice (✓ `safeT_congruence`). All four repairs therefore apply at nested positions, which is where the tool applies them (✓ `repair_narrow_anywhere`, ✓ `repair_guard_anywhere`, ✓ `repair_reorder_anywhere`, ✓ `repair_compensate_anywhere`). The price is that a repair whose soundness depends on the world — guard needs an inert abort world, compensate needs its recovery path safe at the exit points — must have that hypothesis at every world the context can reach, not only at the root, and for the guard that is not free: a validation that validates everywhere, into an idle abort world, forces the abort map to be idempotent on its own image (✓ `global_abort_is_idempotent`). A context-wide guard repair is therefore a repair with a*

single sink, not a fresh abort world per failure. We state the constraint rather than hide it in a hypothesis; the runtime we build (`Abort.v`) instantiates the root-level version.

How a validation is modelled decides whether the repair means anything. The obvious model — a capability that fires only when its predicate holds — makes the repaired protocol *uninhabited* in exactly the worlds the repair addresses: the conformance judgment's action rule asks for a successor, and there is none, so the bridge would hold vacuously there. That is the shape of defect that killed this discipline's predecessor (§13).

Our first attempt to fix it moved the vacuity one constructor along. We sent a failed validation to a *halted* world, one with no successors at all; but the branch a guard protects begins with an action, so the action rule fails there too — and no capability model in this paper has a halted world, since every capability of every model is enabled everywhere (✓ `E2_no_halted`). An abort world is not a world where nothing is enabled. It is a world where everything is enabled and nothing changes. We call such a world *idle*, and *inert* if it is also hazard-free.

That condition is strong and we say plainly what it buys. *Idle* is not "the runtime answers and the answers are errors": it is "the session is over". E_4 (`Abort.v`) is the weaker reading — the validation is a validation at *every* world, the abort flag records the failure, and every other tool keeps working (✓ `E4_validates`, ✓ `E4_no_halted`, ✓ `E4_not_idle`) — and there the same repair on the same protocol fails: the guarded branch is not 1-tolerant (✓ `Gguarded_not_1_tolerant_in_E4`) and the reordered protocol is tolerant at no budget (✓ `reordered_not_tolerant_in_E4`). So the repairs are not about a gate that reports a failure. They are about a gate that *stops the session*, and a deployment that only errors the call does not get the theorem. Both repairs discharge the misselected branch by the same step, so this is one condition, not two.

▶ **Theorem 34** (A guarded branch is futile, not stuck)**.** *A validation of ψ with abort map ab is enabled in every world (✓ `validates_ab_enabled`). Nothing new is reachable from an idle world (✓ `reach_idle`), so every protocol is safe from an inert one (✓ `safeT_inert`); hence a misselected guarded branch, the goal failing at the abort world, is* Futile *— the run is wasted, nothing is harmed — rather than stuck (✓ `guard_abort_is_futile`).*

Every hypothesis of Theorem 34 is discharged concretely (`Abort.v`). E_2 is a runtime with three capabilities — verify, purchase, and a validation of *verified* — all enabled in every world, acting in a live world and behaving as the identity once an abort flag is set. It has inert worlds (✓ `E2_inert_abort`) and no halted ones, and its validation satisfies the aborting model at every live world (✓ `E2_validates`).

▶ **Theorem 35** (Guard)**.** *Let a validate ψ with an inert abort world. A choice with the branch $\ell[\psi].\,a.\,G_\ell$ added is k-safe iff the choice without it is and G_ℓ is k-safe whenever ψ holds (✓ `repair_guard_exact_ab`): a misselected guarded branch diverts to the abort world, so it costs budget but reaches nothing (✓ `guard_absorbs_misselection_ab`). The artifact also carries the blocking-model versions (✓ `repair_guard_exact`); they are the same statement over a validation that cannot fire, and we do not rely on them.*

▶ **Theorem 36** (Reorder)**.** *If chk never creates the hazard and commutes with irr — doing chk then irr re-serializes as irr then chk with the same end world, as STRIPS effects on disjoint variables do — then chk.irr.G is k-safe whenever irr.chk.G is (✓ `repair_reorder_sound`). The two protocols differ precisely at the point of no return: with ψ false, an inert abort world and one hazardous irr-successor, the original is untypable and the reordered protocol is typable at every budget (✓ `repair_reorder_pnr_ab`). Both are also characterized exactly in the blocking model (✓ `reorder_original_exact`, ✓ `reorder_reordered_exact`).*

▶ **Theorem 37** (Compensate). *Appending a recovery path R to a residual G_ℓ is k-safe iff R is safe at every interface point of G_ℓ (✓ `repair_compensate_sound`, an instance of Theorem 30; the converse is not claimed), and it turns a Futile residual Benign as soon as R reaches the goal from one interface point (✓ `repair_compensate_restores_goal`).*

Guard and reorder are exercised end to end on the booking protocol. Reorder: the instance whose fast path purchases before verifying is not 1-tolerant (✓ `Gbad2_not_1_tolerant`); *verify* and *purchase* commute (✓ `commutes_verify_purchase`, proved without functional extensionality by stating the capability context extensionally), and the reordered protocol is tolerant at every k (✓ `Greordered_is_k_tolerant`). Guard, over E_2: the unguarded protocol is not 1-tolerant (✓ `Gbad3_not_1_tolerant`), prefixing the fast branch with the validation makes it tolerant at every k (✓ `Gguarded_is_k_tolerant`), a two-role session *conforms* to the repaired protocol (✓ `Gguarded_inhabited`) — the obligation the blocking model could not meet — and so the bridge applies to it: every run of that session is hazard-free at every budget (✓ `Gguarded_bridge_nonvacuous`).

12 Related Work

Runtime compliance monitoring. The closest work by goal is runtime compliance monitoring of language-model agents, of which TRAC [54] is the most developed: LTL constraints, offline auditing, online monitoring, and predictive and intervening monitors that sample continuations to preempt a violation. We share the question and differ on every mechanism, in two ways. *Prediction versus projection:* TRAC estimates a coming violation by sampling a black box, where the guards fix the intended branch and reachability the consequence of the others, so our prediction is exact and free of any model of the agent, at the price of expressiveness. *The trusted seam:* TRAC's soundness bottoms out in learned labelers whose temporal reasoning its own results show degrading with distance and constraint count; ours in a deterministic checker. The two compose: TRAC's constraints can supply $\mathcal{H}$, its monitor enforce what the condition certified. Choreographies with first-class agent choice mapped to games [56] own the quantitative version of this framing; we are qualitative and static.

MPST with a fault model. Crash-stop failures [8, 9, 10, 47, 48, 49], affine and exceptional sessions [11, 12] and protocol-induced recovery [13] model participants that stop; misselection is one that continues, wrongly, and none classify the consequence or track a goal. Asserted and refined global types [2, 6, 4, 5] supply our syntax; monitoring [3] and blame [7] treat guard violation as an alarm rather than an object of analysis.

Budgeted deviation, elsewhere. A budget on how many times an assumption may be violated is not ours. Reactive synthesis has (k, b)-resilience — tolerate at most k environment-assumption violations, decided on a product with a decrementing counter [42] — and k-robustness for specifications [43]. Robustification computes the largest deviation set a design survives [44, 45], which is our k^* in set currency, and planning has possibilistic K-resilience [26]; closest in fault model, *trembling-hand* LTL_f planning [46] assumes an agent whose action selection is imprecise. We take the mechanism from this literature and claim nothing about it. What is new is the carrier: the deviation is a *typed participant's own branch selection against an asserted guard*, the budget is spent inside a session type, and Theorem 13 carries it through subject reduction to programs, where it distributes over roles (Theorem 17). None of the works above types a program, and none carries a quantity a participant spends across a reduction relation.

Bounded faults and dead ends. Fault-tolerant planning [27, 26] bounds action failures against goal reachability; we bound selections against hazard reachability. Dead-end detection [28, 29], undoability [30, 31] and excuse generation [32] supply the point-of-no-return machinery; reversibility in reinforcement-learning safety [33] shares the intuition without types.

Types and model checkers. Naik and Palsberg [20] and Kobayashi and Ong [21] set the template we instantiate. Resource-aware [22, 23] and graded [24] session types index judgments by a quantity, with the difference drawn in §6. Bounded-model-checking of fault-tolerant algorithms [25] is the verification-side analogue of Theorem 23.

Agents. Formalisms for agents acting on irreversible external state — execution-fidelity invariants [40], capability-containment proofs for skill manifests [41] — guarantee exactly-once effects or capability subsets; neither asks which wrong choice is affordable. Runtime enforcement and provenance gating of tool calls [57, 58, 59, 60], effect-typed agent languages [61], network-enforced session types [19] and LTL trace monitoring [54, 55] act on runs or in the host language; we classify branches before any run exists.

13 Limitations and Conclusion

A design that did not survive. An earlier version of this discipline typed per-role compliance — trust modes, taint propagation, staleness grades — rather than severity. Its mechanized audit found the typing rules vacuous on any protocol containing a capability, its taint update blind to what a capability reads, and its loop condition satisfied by cycles that never reset a grade; the audit is retained in the artifact. The present design has no such premise — deviation lives in the operational semantics as a budget — and that history is why this paper checks its own theorems for vacuity (Theorems 14 and 34).

Scope. The development is mechanized for the head-move semantics, which is the gate's; for ungated deployments Theorem 19 covers every permutation of independent nodes. Asynchronous buffering is outside the synchronous discipline, and a reviewer comparing scope against the mechanised asynchronous subject reduction of [18] will be right to. Guardedness is a hypothesis of progress alone, proved necessary rather than assumed (Theorem 28). On the boolean fragment the tool is cross-checked against a kernel extracted from the proof; the elaboration into its input, environment choices and the widened arithmetic fragment remain trusted. The achievability verdict is about the skill *as written*: an agent that computes the artifact by hand is refuted with it, by design; at the sizes those skills are for that route closes (Finding 5), though the verdict does not claim it always does. The hazard is a declared or derived input; Benign inherits the checker's over-approximation, Catastrophic is a proof. The benchmark is ours, its verdicts stated in advance but authored by us, and only five of its seventeen protocols are in scope for the session-level results; two of its "repairs" are hoists no theorem of ours licenses. The live-agent experiment is small: it establishes non-vacuity, not rates. The guard and reorder repairs need a runtime whose failed validation stops the session, not merely the call (§11). Everything presumes the gated architecture.

Conclusion. A participant that may never err may never act. The question is not whether it will choose wrongly — it will — but whether the protocol survives it: *every affordable mistake is allowed, and every unaffordable one is structurally unreachable.*

References

1 K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *JACM* 63(1), 2016.
2 L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR*, 2010.
3 L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FMOODS/FORTE* 2013; *TCS* 669, 2017.
4 L. Gheri, I. Lanese, N. Sayers, E. Tuosto, N. Yoshida. Design-by-contract for flexible multiparty session protocols. *ECOOP*, 2022.
5 M. Vassor, N. Yoshida. Refinements for multiparty message-passing protocols: specification-agnostic theory and implementation. *ECOOP*, 2024.
6 F. Zhou, F. Ferreira, R. Hu, R. Neykova, N. Yoshida. Statically verified refinements for multiparty protocols. *OOPSLA*, 2020.
7 L. Jia, H. Gommerstadt, F. Pfenning. Monitors and blame assignment for higher-order session types. *POPL*, 2016.
8 A. D. Barwell, A. Scalas, N. Yoshida, F. Zhou. Generalised multiparty session types with crash-stop failures. *CONCUR*, 2022; *LMCS*, 2025.
9 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Designing asynchronous multiparty protocols with crash-stop failures. *ECOOP*, 2023 (Distinguished Paper).
10 K. Peters, U. Nestmann, C. Wagner. Fault-tolerant multiparty session types. *FORTE*, 2022.
11 S. Fowler, S. Lindley, J. G. Morris, S. Decova. Exceptional asynchronous session types. *POPL*, 2019.
12 N. Lagaillardie, R. Neykova, N. Yoshida. Stay safe under panic: affine Rust programming with multiparty session types. *ECOOP*, 2022.
13 R. Neykova, N. Yoshida. Let it recover: multiparty protocol-induced recovery. *CC*, 2017.
14 J. Lange, N. Yoshida. Verifying asynchronous interactions via communicating session automata. *CAV*, 2019.
15 M. R. Clarkson, F. B. Schneider. Hyperproperties. *J. Computer Security* 18(6), 2010.
16 S.-S. Jongmans, F. Ferreira. Synthetic behavioural typing: sound, regular multiparty sessions via implicit local types. *ECOOP*, 2023.
17 D. Castro-Perez, F. Ferreira, S.-S. Jongmans. A synthetic reconstruction of multiparty session types. *PACMPL* 10(POPL):50, 2026.
18 D. Tirore, J. Bengtson, M. Carbone. Multiparty asynchronous session types: a mechanised proof of subject reduction. *ECOOP*, 2025.
19 Larsen, Scalas, Amir, Jacobs, Wagemaker, Foster. NEST: network enforced session types. *ECOOP*, 2026.
20 M. Naik, J. Palsberg. A type system equivalent to a model checker. *ESOP* 2005; *TOPLAS* 30(5), 2008.
21 N. Kobayashi, C.-H. L. Ong. A type system equivalent to the modal mu-calculus model checking of higher-order recursion schemes. *LICS*, 2009.
22 A. Das, J. Hoffmann, F. Pfenning. Work analysis with resource-aware session types. *LICS*, 2018.
23 A. Das, D. Wang, J. Hoffmann. Probabilistic resource-aware session types. *POPL*, 2023.
24 D. Orchard, V.-B. Liepelt, H. Eades III. Quantitative program reasoning with graded modal types. *ICFP*, 2019.
25 I. Stoilkovska, I. Konnov, J. Widder, F. Zuleger. Verifying safety of synchronous fault-tolerant algorithms by bounded model checking. *TACAS*, 2019.
26 D. Aineto, A. Gaudenzi, A. Gerevini, A. Rovetta, E. Scala, I. Serina. Action-failure resilient planning. *ECAI*, 2023.
27 C. Domshlak. Fault tolerant planning: complexity and compilation. *ICAPS*, 2013.
28 M. Steinmetz, J. Hoffmann. Towards clause-learning state space search: learning to recognize dead-ends. *AAAI*, 2016.
29 J. Hoffmann, P. Kissmann, Á. Torralba. "Distance"? Who cares? Tailoring merge-and-shrink heuristics to detect unsolvability. *ECAI*, 2014.

30 J. Daum, Á. Torralba, J. Hoffmann, P. Haslum, I. Weber. Practical undoability checking via contingent planning. *ICAPS*, 2016.
31 J. Med, M. Morak, L. Chrpa, W. Faber. Non-deterministic action reversibility: complexity results. *KR*, 2025.
32 M. Göbelbecker, T. Keller, P. Eyerich, M. Brenner, B. Nebel. Coming up with good excuses: what to do when no plan can be found. *ICAPS*, 2010.
33 A. M. Turner, D. Hadfield-Menell, P. Tadepalli. Conservative agency via attainable utility preservation. *AIES*, 2020.
34 B. Bittner, M. Bozzano, R. Cavada, A. Cimatti, et al. The xSAP safety analysis platform. *TACAS*, 2016.
35 L. Bocchi, J. Lange, E. Tuosto. Three algorithms and a methodology for amending contracts for choreographies. *Sci. Ann. Comp. Sci.* 22(1), 2012.
36 M. Carbone, K. Honda, N. Yoshida. Structured interactional exceptions in session types. *CONCUR*, 2008.
37 P. J. Ramadge, W. M. Wonham. Supervisory control of a class of discrete event processes. *SIAM J. Control Optim.* 25(1), 1987.
38 R. Bloem, B. Könighofer, R. Könighofer, C. Wang. Shield synthesis: runtime enforcement for reactive systems. *TACAS*, 2015.
39 M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 2018.
40 Z. Yin. Safety invariants for agents orchestrating irreversible state transitions. arXiv:2608.00783, 2026.
41 A. Metere. Methods for formal verification of agent skills: three layers toward a mechanically checkable capability-containment proof. arXiv:2605.23951, 2026.
42 R. Ehlers, U. Topcu. Resilience to intermittent assumption violations in reactive synthesis. *HSCC*, 2014.
43 R. Bloem, K. Chatterjee, K. Greimel, T. A. Henzinger, B. Jobstmann. Synthesizing robust systems. *FMCAD* 2009; *Acta Informatica* 51, 2014.
44 C. Zhang, D. Garlan, E. Kang. A behavioral notion of robustness for software systems. *FSE*, 2020.
45 R. Meira-Góes, I. Dardik, E. Kang, S. Lafortune, S. Tripakis. Safe environmental envelopes of discrete systems. *CAV*, 2023.
46 P. Yu, S. Zhu, G. De Giacomo, M. Kwiatkowska, M. Y. Vardi. The trembling-hand problem for LTLf planning. *IJCAI*, 2024.
47 M. Viering, R. Hu, P. Eugster, L. Ziarek. A multiparty session typing discipline for fault-tolerant event-driven distributed programming. *PACMPL* 5(OOPSLA), 2021.
48 M. A. Le Brun, O. Dardha. MAGπ: types for failure-prone communication. *ESOP*, 2023.
49 P. Hou, N. Lagaillardie, N. Yoshida. Fearless asynchronous communications with timed multiparty session protocols. *ECOOP*, 2024.
50 L. Gheri, N. Yoshida. Hybrid multiparty session types: compositionality for protocol specifications. *PACMPL* 7(OOPSLA), 2023.
51 F. Barbanera, I. Lanese, E. Tuosto. Composing communicating systems, synchronously. *JLAMP*, 2021.
52 S. Basu, T. Bultan. Automated choreography repair. *FASE*, 2016.
53 I. Lanese, F. Montesi, G. Zavattaro. Amending choreographies. *WWV*, 2013.
54 P. A. Alamdari, T. Q. Klassen, S. A. McIlraith. Formal methods meet LLMs: auditing, monitoring, and intervention for compliance of advanced AI systems. *FAccT*, 2026.
55 L. Elkoussy, J. Perez. AgentLTL: a trace-verification framework for measuring, enforcing, and training procedural compliance in tool-using LLM agents. *arXiv:2607.02599*, 2026.
56 K. Gopinathan, J. Feser, M. Naim, Z. Tavares, E. Bingham. Pact: a choreographic language for agentic ecosystems. *2nd International Workshop on Choreographic Programming*, 2026; *arXiv:2605.03143*.
57 H. Wang, C. M. Poskitt, et al. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE*, 2026.
58 E. Debenedetti et al. Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*, 2025.

59 M. Costa et al. Securing AI agents with information-flow control (FIDES). *arXiv:2505.23643*, 2025.

60 T. Shi et al. Progent: securing AI agents with privilege control. *arXiv:2504.11703*, 2025.

61 H. Tan, Y. Wang, P. Zhang, S. Li, J. Shen. ETAS: an effect-typed language for agent systems. *arXiv:2607.17780*, 2026.